

PVN3D: A Deep Point-wise 3D Keypoints Voting Network for 6DoF Pose Estimation

Yisheng He¹ Wei Sun² Haibin Huang³ Jianran Liu² Haoqiang Fan² Jian Sun²

¹Hong Kong University of Science and Technology

²Megvii Inc.

³Kuaishou Technology

Abstract

In this work, we present a novel data-driven method for robust 6DoF object pose estimation from a single RGBD image. Unlike previous methods that directly regressing pose parameters, we tackle this challenging task with a keypoint-based approach. Specifically, we propose a deep Hough voting network to detect 3D keypoints of objects and then estimate the 6D pose parameters within a least-squares fitting manner. Our method is a natural extension of 2D-keypoint approaches that successfully work on RGB based 6DoF estimation. It allows us to fully utilize the geometric constraint of rigid objects with the extra depth information and is easy for a network to learn and optimize. Extensive experiments were conducted to demonstrate the effectiveness of 3D-keypoint detection in the 6D pose estimation task. Experimental results also show our method outperforms the state-of-the-art methods by large margins on several benchmarks. Code and video are available at <https://github.com/ethnhe/PVN3D.git>.

1. INTRODUCTION

In this paper, we study the problem of 6DoF pose estimation, i.e. recognize the 3D location and orientation of an object in a canonical frame. It is an important component in many real-world applications, such as robotic grasping and manipulation [6, 48, 56], autonomous driving [11, 5, 53], augmented reality [31] and so on.

6DoF estimation has been proven a quite challenging problem due to variations of lighting, sensor noise, occlusion of scenes and truncation of objects. Traditional methods like [19, 30] used hand-crafted features to extract the correspondence between images and object mesh models. Such empirical human-designed features would suffer from limited performance with changing illumination conditions

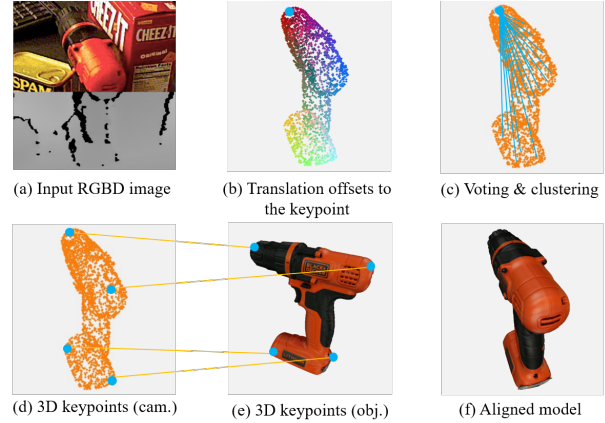


Figure 1. **Pipeline of PVN3D:** With an input RGBD image (a), we use a deep Hough voting network to predict the per-point translation offset to the selected keypoint (b). Each point on the same object votes for the selected keypoint and the center of the cluster is selected as a predicted keypoint (c). A least-squares fitting method is then applied to estimate 6D pose parameters (d)-(e). The model transformed by estimated pose parameters is shown in Figure (f).

and scenes with heavy occlusion. More recently, with the explosive growth of machine learning and deep learning techniques, Deep Neural Network (DNN) based methods have been introduced into this task and reveal promising improvements. [50, 52] proposed to regress rotation and translation of objects directly with DNNs. However, these methods usually had poor generalization due to the non-linearity of the rotation space explained by [37]. Instead, recent works utilized DNNs to detect 2D keypoints of an object, and computed 6D pose parameters with Perspective-n-Point (PnP) algorithms [37, 36, 41, 47]. Although these two-stage approaches performed more stable, most of them were built on top of the 2D projection. Errors that are small in projection can be large in real 3D space. Also, different keypoints in 3D space may be overlapped after 2D projection, making them hard to be distinguished. Moreover, geometric constraint information of rigid objects would be

This work is supported by The National Key Research and Development Program of China (2018YFC0831700).

partially lost due to projection.

On the other hand, with the development of inexpensive RGBD sensors, more and more RGBD datasets are available. The extra depth information allows 2D algorithms to be extended into 3D space with better performance, like PointFusion [53], Frustum pointnets[39] and VoteNet[38]. Towards this end, we extend 2D-keypoint-based approaches to 3D keypoint to fully utilize geometric constraint information of rigid objects and significantly improved the accuracy of 6DoF estimation. More specifically, we develop a deep 3D keypoints Hough voting neural network to learn the point-wise 3D offset and vote for 3D keypoints, as shown in Figure 1. Our key observation is a simple geometric property that positional relationship between two points of a rigid object in 3D space is fixed. Hence, given a visible point on the object surface, its coordinate and orientation can be obtained from depth images and its translation offset to selected keypoint is also fixed and learnable. Meanwhile, learning point-wise Euclidean offset is straightforward for network and easier to optimize.

To handle scenes with multiple objects, we also introduce an instance semantic segmentation module into the network and jointly optimized with keypoint voting. We find that jointly training these tasks boosts the performance of each other. Specifically, semantic information improves translation offset learning by identifying which part a point belongs to and the size information contained in translation offsets helps the model to distinguish objects with similar appearance but different size.

We further conduct experiments on YCB-Video and LineMOD datasets to evaluate our method. Experimental results show that our approach outperforms current state-of-the-art methods by a significant margin.

To summarize, the main contributions of this work are as follows:

- A novel deep 3D keypoints Hough voting network with instance semantic segmentation for 6DoF Pose Estimation of single RGBD image.
- State-of-the-art 6DoF pose estimation performance on YCB and LineMOD datasets.
- An in-depth analysis of our 3D-keypoint-based method and comparison with previous approaches, demonstrating that 3D-keypoint is a key factor to boost performance for 6DoF pose estimation. We also show that jointly training 3D-keypoint and semantic segmentation can further improve the performance.

2. Related Work

2.1. Holistic Methods

Holistic methods directly estimate the 3D position and orientation of objects in a given image. Classical template-

based methods construct rigid templates and scan through the image to compute the best matched pose [21, 13, 17]. Such templates are not robust to clustered scenes. Recently, some Deep Neural Network (DNN) based methods are proposed to directly regress the 6D pose of cameras or objects [52, 50, 14]. However, non-linearity of the rotation space making the data-driven DNN hard to learn and generalize. To address this problem, some approaches use post-refinement procedure [26, 50] to refine the pose iteratively, others discretize the rotation space and simplify it to be a classification problem [49, 43, 45]. For the latter approach, post-refinement processes are still required to compensate for the accuracy sacrificed by the discretization.

2.2. Keypoint-based Methods

Current keypoint-based methods first detect the 2D keypoints of an object in the images, then utilize a PnP algorithm to estimate the 6D pose. Classical methods [30, 42, 2] are able to detect 2D keypoint of objects with rich texture efficiently. However, they can not handle texture-less objects. With the development of deep learning techniques, some neural-network-based 2D keypoints detection methods are proposed. [41, 47, 20] directly regress the 2D coordinate of the keypoints, while [33, 24, 34] use heatmaps to locate the 2D keypoints. To better deal with truncated and occluded scenes, [37] proposes a pixel-wise voting network to vote for the 2D keypoints location. These 2D keypoint based methods aim to minimize the 2D projection errors of objects. However, errors that are small in projection may be large in the real 3D world. [46] extracts 3D keypoints from two views of synthetic RGB images to recover 3D poses. Nevertheless, they only utilize the RGB images, on which geometric constraint information of rigid objects partly lost due to projection, and different keypoints in 3D space may be overlapped and hard to be distinguished after projected to 2D. The advent of cheap RGBD sensors enables us to do everything in 3D with captured depth images.

2.3. Dense Correspondence Methods

These approach utilize Hough voting scheme [28, 44, 12] to vote for final results with per-pixel prediction. They either use random forest [3, 32] or CNNs [23, 9, 27, 35, 51] to extract feature and predict the corresponding 3D object coordinates for each pixel and then vote for the final pose results. Such dense 2D-3D correspondence making these methods robust to occluded scenes, while the output space is quite large. PVNet [37] uses per-pixel voting for 2D Keypoints to combine the advantages of Dense methods and keypoint-based methods. We further extend this method to 3D keypoints with extra depth information and fully utilize geometric constraints of rigid objects.

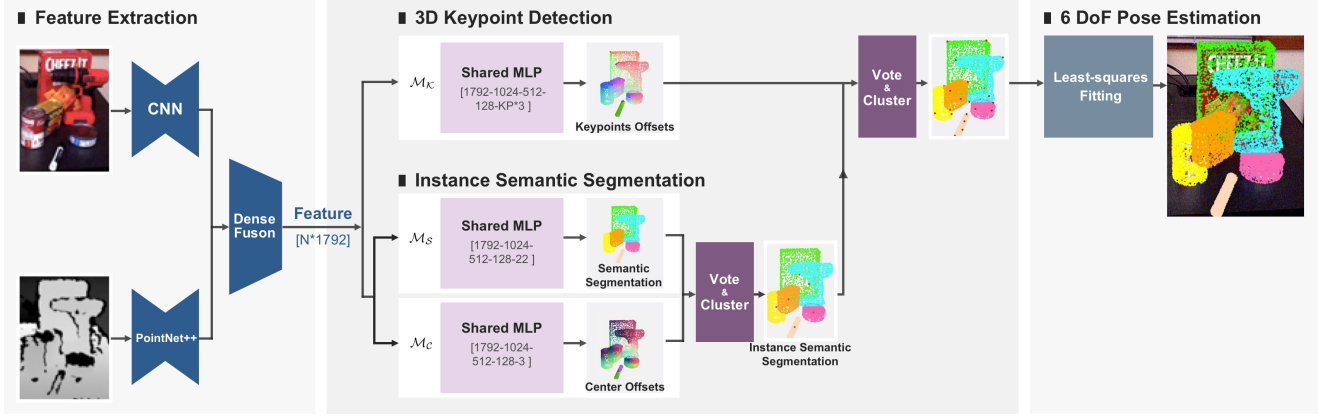


Figure 2. **Overview of PVN3D.** The Feature Extraction module extracts the per-point feature from an RGBD image. They are fed into module $\mathcal{M}_K$, $\mathcal{M}_C$ and $\mathcal{M}_S$ to predict the translation offsets to keypoints, center point and semantic labels of each point respectively. A clustering algorithm is then applied to distinguish different instances with the same semantic label and points on the same instance vote for their target keypoints. Finally, a least-square fitting algorithm is applied to the predicted keypoints to estimate 6DoF pose parameters.

3. Proposed Method

Given an RGBD image, the task of 6DoF pose estimation is to estimate the rigid transformation that transforms an object from its object world coordinate system to the camera world coordinate system. Such transformation consists of a 3D rotation $\mathbf{R} \in SO(3)$ and a translation $\mathbf{t} \in \mathbb{R}^3$.

3.1. Overview

To tackle this task, we develop a novel approach based on a deep 3D Hough voting network, as shown in Figure 2. The proposed method is a two-stage pipeline with 3D keypoint detection followed by a pose parameters fitting module. More specifically, taking an RGBD image as input, a feature extraction module would be used to fuse the appearance feature and geometry information. The learned feature would be fed into a 3D keypoint detection module $\mathcal{M}_K$ which was trained to predict the per-point offsets w.r.t keypoints. Additionally, we include an instance segmentation module for multiple objects handling where a semantic segmentation module $\mathcal{M}_S$ predicts the per-point semantic label, and a center voting module $\mathcal{M}_C$ predicts the per-point offsets to object center. With the learned per-point offset, the clustering algorithm [7] is applied to distinguish different instances with the same semantic label and points on the same instance vote for their target keypoints. Finally, a least-square fitting algorithm is applied to the predicted keypoints to estimate 6DoF pose parameters.

3.2. Learning Algorithm

The goal of our learning algorithm is to train a 3D keypoint detection module $\mathcal{M}_K$ for offset prediction as well as a semantic segmentation module $\mathcal{M}_S$ and center voting module $\mathcal{M}_C$ for instance-level segmentation. This naturally makes training our network multi-task learning, which is

achieved by a supervised loss we designed and several training details we adopt.

3D Keypoints Detection Module. As shown in Figure 2, with the per-point feature extracted by the feature extraction module, a 3D keypoint detection module $\mathcal{M}_K$ is used to detect the 3D keypoints of each object. To be specific, $\mathcal{M}_K$ predicts the per-point Euclidean translation offset from visible points to target keypoints. These visible points, together with the predicted offsets then vote for the target keypoints. The voted points are then gathered by clustering algorithms and centers of clusters are selected as the voted keypoints.

We give a deeper view of $\mathcal{M}_K$ as follows. Given a set of visible seed points $\{p_i\}_{i=1}^N$ and a set of selected keypoints $\{kp_j\}_{j=1}^M$ belonging to the same object instance I , we denote $p_i = [x_i; f_i]$ with x_i the 3D coordinate and f_i the extracted feature. We denote $kp_j = [y_j]$ with y_j the 3D coordinate of the keypoint. $\mathcal{M}_K$ absorbs feature f_i of each seed point and generates translation offset $\{of_i^j\}_{j=1}^M$ for them, where of_i^j denotes the translation offset from the i_{th} seed point to the j_{th} keypoint. Then the voted keypoint can be denoted as $vkp_i^j = x_i + of_i^j$. To supervise the learning of of_i^j , we apply an L1 loss:

$$L_{\text{keypoints}} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M \|of_i^j - of_i^{j*}\| \mathbb{I}(p_i \in I) \quad (1)$$

where of_i^{j*} is the ground truth translation offset; M is the total number of selected target keypoints; N is the total number of seeds and $\mathbb{I}$ is an indicating function equates to 1 only when point p_i belongs to instance I , and 0 otherwise.

Instance Semantic Segmentation Module. To handle scenes with multi objects, previous methods [50, 53, 39] utilize existing detection or semantic segmentation architecture to pre-process the image and obtain RoIs (regions

of interest) containing only single objects. Then build the pose estimation models with the extracted ROIs as input to simplify the problem. However, as we have formulated the pose estimation problem to first detect keypoints of objects with a translation offsets to keypoints learning module, we believe that the two tasks can enhance the performance of each other. On the one hand, the semantic segmentation module forces the model to extract global and local features on instance to distinguish different objects, which helps to locate a point on the object and does good to the keypoint offset reasoning procedure. On the other hand, size information learned for the prediction of offsets to the keypoints helps distinguish objects with similar appearance but different in size. Under such observation, we introduce a point-wise instance semantic segmentation module $\mathcal{M}_S$ into the network and jointly optimized it with module $\mathcal{M}_K$.

To be specific, given the per-point extracted feature, the semantic segmentation module $\mathcal{M}_S$ predicts the per-point semantic labels. We supervise this module with Focal Loss [29]:

$$L_{semantic} = -\alpha(1 - q_i)^\gamma \log(q_i) \quad (2)$$

where $q_i = c_i \cdot l_i$

with α the α -balance parameter, γ the focusing parameter, c_i the predicted confidence for the i_{th} point belongs to each class and l_i the one-hot representation of ground true class label.

Meanwhile, the center voting module $\mathcal{M}_C$ is applied to vote for centers of different object so as to distinguish different instance. We propose such module under the inspiration of CenterNet [10] but further extend the 2D center point to 3D. Compared to 2D center points, different center points in 3D won't suffer from occlusion due to camera projection in some viewpoints. Since we can regard the center point as a special keypoint of an object, module $\mathcal{M}_C$ is similar to the 3D keypoint detection module $\mathcal{M}_K$. It takes in the per-point feature but predicts the Euclidean translation offset Δx_i to the center of objects it belongs to. The learning of Δx_i is also supervised by an L1 loss:

$$L_{center} = \frac{1}{N} \sum_{i=1}^N \|\Delta x_i - \Delta x_i^*\| \mathbb{I}(p_i \in I) \quad (3)$$

where N denotes the total number of seed points on the object surface and Δx_i^* is the ground truth translation offset from seed p_i to the instance center. $\mathbb{I}$ is an indication function indicating whether point p_i belongs to that instance.

Multi-task loss. We supervise the learning of module $\mathcal{M}_K$, $\mathcal{M}_S$ and $\mathcal{M}_C$ jointly with a multi-tasks loss:

$$L_{multi-task} = \lambda_1 L_{keypoints} + \lambda_2 L_{semantic} + \lambda_3 L_{center} \quad (4)$$

where λ_1 , λ_2 and λ_3 are the weights for each task. Experimental results shows that jointly training these tasks boosts the performance of each other.

3.3. Training and Implementation

Network Architecture. The first part in Figure 2 is a feature extraction module. In this module, a PSPNet [55] with an ImageNet [8] pretrained ResNet34 [16] is applied to extract the appearance information in RGB images. A PointNet++ [40] extracts the geometry information in point clouds and their normal maps. They are further fused by a DenseFusion block [50] to obtain the combined feature for each point. After the process of this module, each point p_i has a feature $f_i \in \mathbb{R}^C$ of C dimension. The following module $\mathcal{M}_K$, $\mathcal{M}_S$ and $\mathcal{M}_C$ are composed of shared Multi-Layer Perceptrons (MLPs) shown in Figure 2. We sample $N = 12288$ points (pixels) for each frame of RGBD image and set $\lambda_1 = \lambda_2 = \lambda_3 = 1.0$ in Formula 4.

Keypoint Selection. The 3D keypoints are selected from 3D object models. In 3D object detection algorithms [39, 53, 38], eight corners of the 3D bounding box are selected. However, These bounding box corners are virtual points that are far away from points on the object, making point-based networks difficult to aggregate scene context in the vicinity of them. The longer distance to the object points results in larger localization errors, which may do harm to the compute of 6D pose parameters. Instead, points selected from the object surface will be quite better. Therefore, we follow [37] and use the farthest point sampling (FPS) algorithm to select keypoints on the mesh. Specifically, we initial the selection procedure by adding the center point of the object model in an empty keypoint set. Then update it by adding a new point on the mesh that is farthest to all the selected keypoints repeatedly, until M keypoints are obtained.

Least-Squares Fitting. Given two point sets of an object, one from the M detected keypoints $\{kp_j\}_{j=1}^M$ in the camera coordinate system, and another from their corresponding points $\{kp'_j\}_{j=1}^M$ in the object coordinate system, the 6D pose estimation module computes the pose parameters (R, t) with a least-squares fitting algorithm [1], which finds R and t by minimizing the following square loss:

$$L_{least-squares} = \sum_{j=1}^M \|kp_j - (R \cdot kp'_j + t)\|^2 \quad (5)$$

where M is the number of selected keypoints of a object.

4. Experiments

4.1. Datasets

We evaluate our method on two benchmark datasets.

YCB-Video Dataset contains 21 YCB [4] objects of varying shape and texture. 92 RGBD videos of the subset of objects were captured and annotated with 6D pose and instance semantic mask. The varying lighting conditions, significant image noise, and occlusions make this dataset chal-

	Without Iterative Refinement						With Iterative Refinement					
	PoseCNN[52]		DF(per-pixel)[50]		PVN3D		PoseCNN+ICP[52]		DF(Iterative)[50]		PVN3D+ICP	
	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)	ADD(S)
002_master_chef_can	83.9	50.2	95.3	70.7	96.0	80.5	95.8	68.1	96.4	73.2	95.2	79.3
003_cracker_box	76.9	53.1	92.5	86.9	96.1	94.8	92.7	83.4	95.8	94.1	94.4	91.5
004_sugar_box	84.2	68.4	95.1	90.8	97.4	96.3	98.2	97.1	97.6	96.5	97.9	96.9
005_tomato_soup_can	81.0	66.2	93.8	84.7	96.2	88.5	94.5	81.8	94.5	85.5	95.9	89.0
006_mustard_bottle	90.4	81.0	95.8	90.9	97.5	96.2	98.6	98.0	97.3	94.7	98.3	97.9
007_tuna_fish_can	88.0	70.7	95.7	79.6	96.0	89.3	97.1	83.9	97.1	81.9	96.7	90.7
008_pudding_box	79.1	62.7	94.3	89.3	97.1	95.7	97.9	96.6	96.0	93.3	98.2	97.1
009_gelatin_box	87.2	75.2	97.2	95.8	97.7	96.1	98.8	98.1	98.0	96.7	98.8	98.3
010_potted_meat_can	78.5	59.5	89.3	79.6	93.3	88.6	92.7	83.5	90.7	83.6	93.8	87.9
011_banana	86.0	72.3	90.0	76.7	96.6	93.7	97.1	91.9	96.2	83.3	98.2	96.0
019_pitcher_base	77.0	53.3	93.6	87.1	97.4	96.5	97.8	96.9	97.5	96.9	97.6	96.9
021_bleach_cleanser	71.6	50.3	94.4	87.5	96.0	93.2	96.9	92.5	95.9	89.9	97.2	95.9
024_bowl	69.6	69.6	86.0	86.0	90.2	90.2	81.0	81.0	89.5	89.5	92.8	92.8
025_mug	78.2	58.5	95.3	83.8	97.6	95.4	94.9	81.1	96.7	88.9	97.7	96.0
035_power_drill	72.7	55.3	92.1	83.7	96.7	95.1	98.2	97.7	96.0	92.7	97.1	95.7
036_wood_block	64.3	64.3	89.5	89.5	90.4	90.4	87.6	87.6	92.8	92.8	91.1	91.1
037_scissors	56.9	35.8	90.1	77.4	96.7	92.7	91.7	78.4	92.0	77.9	95.0	87.2
040_large_marker	71.7	58.3	95.1	89.1	96.7	91.8	97.2	85.3	97.6	93.0	98.1	91.6
051_large_clamp	50.2	50.2	71.5	71.5	93.6	93.6	75.2	75.2	72.5	72.5	95.6	95.6
052_extra_large_clamp	44.1	44.1	70.2	70.2	88.4	88.4	64.4	64.4	69.9	69.9	90.5	90.5
061_foam_brick	88.0	88.0	92.2	92.2	96.8	96.8	97.2	97.2	92.0	92.0	98.2	98.2
ALL	75.8	59.9	91.2	82.9	95.5	91.8	93.0	85.4	93.2	86.1	96.1	92.3

Table 1. Quantitative evaluation of 6D Pose (ADD-S AUC [52], ADD(S) AUC [19]) on the YCB-Video Dataset. Symmetric objects’ names are in bold.

		w/o iter. ref.		w/ iter. ref.	
		DF(p.p.)	PVN3D	DF(iter.)	PVN3D+ICP
large_clamp	ADD-S	87.7	93.9	90.3	96.2
extra_large_clamp	ADD-S	75.0	90.1	74.9	93.6
ALL	ADD-S	93.3	95.7	94.8	96.4
	ADD(S)	84.9	91.9	89.4	92.7

Table 2. Quantitative evaluation results on the YCB-Video dataset with ground truth instance semantic segmentation result.

lensing. We follow [52] and split the dataset into 80 videos for training and another 2,949 keyframes chosen from the rest 12 videos for testing. Following [52], we add the synthetic images into our training set. A hole completion algorithm [25] is also applied to improve the quality of depth images.

LineMOD Dataset [18] consists of 13 low-textured objects in 13 videos, annotated 6D pose and instance mask. The main challenge of this dataset is the cluttered scenes, texture-less objects, and lighting variations. We follow prior works [52] to split the training and testing set. Also, we follow [37] and add synthesis images into our training set.

4.2. Evaluation Metrics

We follow [52] and evaluate our method with the average distance ADD and ADD-S metric [52]. The average distance ADD metric [19] evaluates the mean pair-wise distance between object vertexes transformed by the predicted

6D pose $[R, t]$ and the ground true pose $[R^*, t^*]$:

$$\text{ADD} = \frac{1}{m} \sum_{x \in \mathcal{O}} \|(Rx + t) - (R^*x + t^*)\| \quad (6)$$

where x is a vertex of totally m vertexes on the object mesh $\mathcal{O}$. The ADD-S metric is designed for symmetric objects and the mean distance is computed based on the closest point distance:

$$\text{ADD-S} = \frac{1}{m} \sum_{x_1 \in \mathcal{O}} \min_{x_2 \in \mathcal{O}} \|(Rx_1 + t) - (R^*x_2 + t^*)\| \quad (7)$$

For evaluation, we follow [52, 50] and compute the ADD-S AUC, the area under the accuracy-threshold curve, which is obtained by varying the distance threshold in evaluation. The ADD(S)[19] AUC is computed in a similar way but calculate ADD distance for non-symmetric objects and ADD-S distance for symmetric objects.

4.3. Evaluation on YCB-Video & LineMOD Dataset

Table 1 shows the evaluation results for all the 21 objects in the YCB-Video dataset. We compare our model with other single view methods. As shown in the Table, our model without any iterative refinement procedure (PVN3D) surpasses all other approaches by a large margin, even when they are iterative refined. On the ADD(S) metric, our model outperforms PoseCNN+ICP [52] by 6.4%

	RGB			RGBD					
	PoseCNN DeepIM [26, 52]	PVNet [37]	CDPN [27]	Implicit ICP[45]	SSD-6D ICP[22]	Point- Fusion[50]	DF(per- pixel)[50]	DF(ite- rative)[50]	PVN3D
ape	77.0	43.6	64.4	20.6	65.0	70.4	79.5	92.3	97.3
benchvise	97.5	99.9	97.8	64.3	80.0	80.7	84.2	93.2	99.7
camera	93.5	86.9	91.7	63.2	78.0	60.8	76.5	94.4	99.6
can	96.5	95.5	95.9	76.1	86.0	61.1	86.6	93.1	99.5
cat	82.1	79.3	83.8	72.0	70.0	79.1	88.8	96.5	99.8
driller	95.0	96.4	96.2	41.6	73.0	47.3	77.7	87.0	99.3
duck	77.7	52.6	66.8	32.4	66.0	63.0	76.3	92.3	98.2
eggbox	97.1	99.2	99.7	98.6	100.0	99.9	99.9	99.8	99.8
glue	99.4	95.7	99.6	96.4	100.0	99.3	99.4	100.0	100.0
holepuncher	52.8	82.0	85.8	49.9	49.0	71.8	79.0	92.1	99.9
iron	98.3	98.9	97.9	63.1	78.0	83.2	92.1	97.0	99.7
lamp	97.5	99.3	97.9	91.7	73.0	62.3	92.3	95.3	99.8
phone	87.7	92.4	90.8	71.0	79.0	78.8	88.0	92.8	99.5
ALL	88.6	86.3	89.9	64.7	79.0	73.7	86.2	94.3	99.4

Table 3. Quantitative evaluation of 6D Pose on ADD(S) [19] metric on the LineMOD dataset. Objects with bold name are symmetric.

	DF(RT)[50]	DF(3D KP)[50]	Ours(RT)	Ours(2D KPC)	Ours(2D KP)	PVNet[37]	Ours(Corr)	Ours(3D KP)
ADD-S	92.2	93.1	92.8	78.2	81.8	-	92.8	95.5
ADD(S)	86.9	87.9	87.3	73.8	77.2	73.4	88.1	91.8

Table 4. Quantitative evaluation of 6D Poses on the YCB-Video dataset with different formulations. All with our predicted segmentation.

	VoteNet[38]	BBox 8	FPS 4	FPS 8	FPS 12
ADD-S	89.9	94.0	94.3	95.5	94.5
ADD(S)	85.1	90.2	90.5	91.8	90.7

Table 5. Effect of different keypoint selection methods of PVN3D. Results of VoteNet[38], another 3D bounding box detection approach are added as a simple baseline to compare with our BBox8.

and exceeds DF(Iterative) [50] by 5.7%. With iterative refinement, our model (PVN3D+ICP) achieves even better performance. Note that one challenge of this dataset is to distinguish the large clamp and extra-large clamp, on which previous methods [50, 52] suffer from poor detection results. We also report evaluation results with ground truth segmentation in Table 2, which shows that our PVN3D still achieves the best performance. Some Qualitative results are shown in Figure 3. Table 3 demonstrates the evaluation results on LineMOD dataset. Our model also achieves the best performance.

Robust to Occlusion Scenes. One of the biggest advantages of our 3D-keypoint-based method is that it’s robust to occlusion naturally. To explore how different methods are influenced by different degrees of occlusion, we follow [50] and calculate the percentage of invisible points on the object surface. Accuracy of ADD-S < 2cm under different invisible surface percentage is shown in Figure 4. The performance of different approaches is very close when 50% of points are invisible. However, with the percentage of invisible part increase, DenseFusion and PoseCNN+ICP fall faster comparing with ours. Figure 3 shows that our model performs well even when objects are heavily occluded.

4.4. Ablation Study

In this part, we explore the influence of different formulation for 6DoF pose estimation and the effect of keypoint

selection methods. We also probe the effect of multi-task learning.

Comparisons to Directly Regressing Pose. To compare our 3D keypoint based formulation with formulations that directly regressing the 6D pose parameters $[R, t]$ of an object, we simply modify our 3D keypoint voting module $\mathcal{M}_K$ to directly regress the quaternion rotation R and the translation parameters t for each point. We also add a confidence header following DenseFusion [50] and select the pose with the highest confidence as the final proposed pose. We supervise the training process using ShapeMatch-Loss [52] with confidence regularization term [50] following DenseFusion. Experimental results in Table 4 shows that our 3D keypoint formulation performs quite better.

To eliminate the influence of different network architecture, we also modify the header of DenseFusion(per-pixel) to predict the per-point translation offset and compute the 6D pose following our keypoint voting and least-squares fitting procedure. Table 4 reveals that the 3D keypoint formulation, DF(3D KP) in the Table, performs better than the RT regression formulation, DF(RT). That’s because the 3D keypoint offset search space is smaller than the non-linearity of rotation space, which is easier for neural networks to learn, enabling them to be more generalizable.

Comparisons to 2D Keypoints. In order to contrast the influence of 2D and 3D keypoints, we project the voted 3D keypoints back to 2D with the camera intrinsic parameters. A PnP algorithm with Random Sample Consensus (RANSAC) is then applied to compute the 6D pose parameters. Table 4 shows that algorithms with 3D keypoint formulation, denoted as Ours(3D KP) in the table, outperforms 2D keypoint, denoted Ours(2D KP) in the table, by 13.7% under ADD-S metric. That’s because PnP algorithms aim to minimize the projection error. However, pose estimation

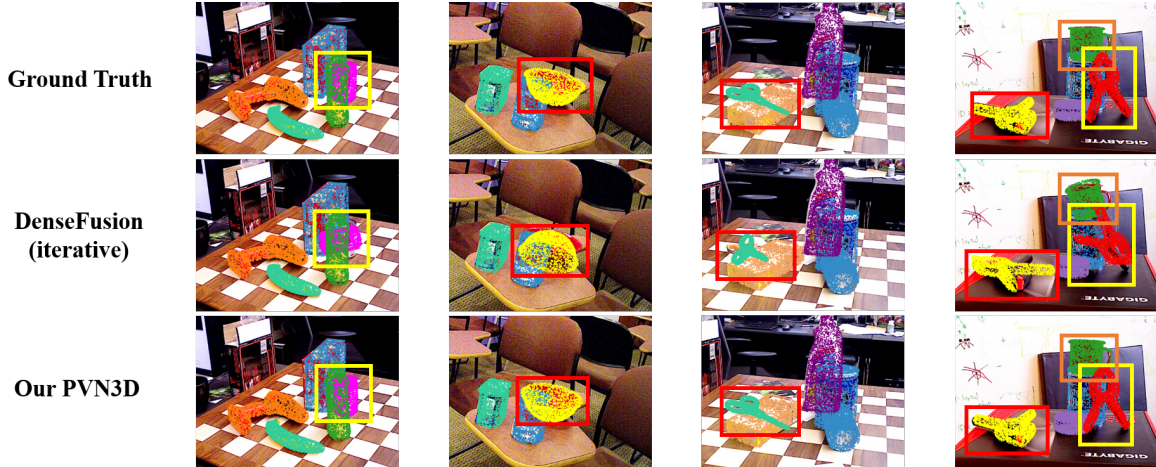


Figure 3. **Qualitative results on the YCB-Video dataset.** Points on different meshes in the same scene are in different colors. They are projected back to the image after being transformed by the predicted pose. We compare our PVN3D **without any iterative refinement procedure** to DenseFusion with iterative refinement (2 iterations). Our model distinguishes the challenging large clamp and extra-large clamp and estimates their poses well. Our model is also robust in heavily occluded scenes.

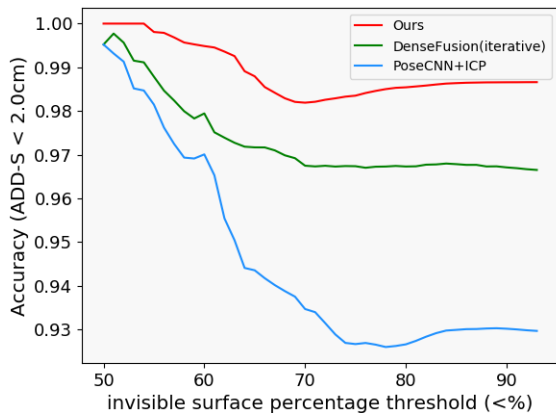


Figure 4. Performance of different approaches under increasing levels of occlusion on the YCB-Video dataset.

errors that are small in projection may be quite large in the 3D real world.

To compare the influence between 2D and 3D center point in our instance semantic segmentation module, we also project our voted 3D center point to 2D in the Instance Semantic Segmentation module(Ours(2D KPC)). We apply a similar Mean Shift algorithm to cluster the voted 2D center points to distinguish different instance, finding that in occlusion scenes, different instances are hard to be differentiated when their centers are close to each other after projected on 2D, while they are far away from each other and can be easily differentiated in 3D real world. Note that other existing 2D keypoints detection approaches, such as heatmap [33, 24, 34] and vector voting [37] models may also suffer from overlapped keypoints. By definition, centers of most objects in our daily life won't be overlapped as they

usually lie within the object while they may be overlapped after projected to 2D. In a word, the object world is in 3D, we believe that building models on 3D is quite important.

Comparisons to Dense Correspondence Exploring.

We modify our 3D keypoint offset module $\mathcal{M}_K$ to output the corresponding 3D coordinate of each point in the object coordinate system and apply the least-squares fitting algorithm to compute the 6DoF pose. An L1 loss similar to Formula 3 is applied to supervise the training of the corresponding 3D coordinate. Evaluation results are shown as Ours(corr) in Tabel 4, which shows that our 3D keypoints formulation still performs quite better. We believe that regressing object coordinates is more difficult than keypoint detection. Because the model has to recognize each point of a mesh in the image and memorize its coordinate in the object coordinate system. However, detecting keypoints on objects in the camera system is easier since many keypoints are visible and the model can aggregate scene context in the vicinity of them.

Effect of 3D Keypoints Selection. In this part, we select 8 corners of the 3D bounding box and compares them with points selected from the FPS algorithm. Different number of keypoints generated by FPS are also taken into consideration. Table 5 shows that keypoints selected by the FPS algorithm on the object enable our model to perform better. That's because the bounding box corners are virtual points that are far away from points on the object. Therefore, point-based networks are difficult to aggregate scene context in the vicinity of these virtual corner points. Also, 8 keypoints selected from FPS algorithm is a good choice for our network to learn. More keypoints may better eliminate errors when recovering pose in the least-squares fitting module, but harder for the network to learn as the output

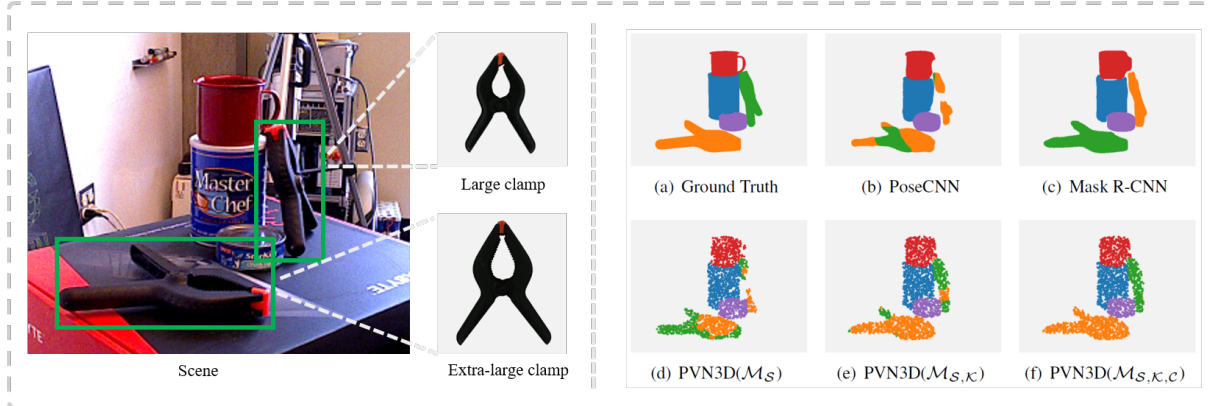


Figure 5. **Qualitative results of semantic segmentation on the challenging YCB-Video dataset.** (a) shows the ground truth label. Different objects are labeled in different colors, with large clamp colored green and extra-large clamp colored orange. In (b)-(c), the simple baselines PoseCNN[52] and Mask R-CNN [15] are confused by the two objects. In (d), our semantic segmentation module $\mathcal{M}_S$, trained separately, can not distinguish them well either. In (e), jointly training $\mathcal{M}_S$ with the keypoints offset voting module $\mathcal{M}_K$ performs better. In (f), with the voted center and Mean-Shift clustering algorithm, our model can distinguish them well.

	$\mathcal{M}_K$ +MRC	$\mathcal{M}_K$ +GT	$\mathcal{M}_{K,S}$ +GT	$\mathcal{M}_{K,S,C}$	$\mathcal{M}_{K,S,C}$ +GT
ADD-S	93.5	94.8	95.2	95.5	95.7
ADD(S)	89.7	90.6	91.3	91.8	91.9

Table 6. Performance of PVN3D with different instance semantic segmentation on all objects in the YCB-Video dataset. $\mathcal{M}_K$, $\mathcal{M}_S$ and $\mathcal{M}_C$ denote keypoint offset module, semantic segmentation and center point offset module of PVN3D respectively. +MRC and +GT denotes inference with segmentation result of Mask R-CNN and ground truth segmentation respectively.

	PoseCNN [52]	Mask R- CNN[15]	PVN3D ($\mathcal{M}_S$)	PVN3D ($\mathcal{M}_{S,K}$)	PVN3D ($\mathcal{M}_{S,K,C}$)
large clamp	43.1	48.4	58.6	62.5	70.2
extra-large clamp	30.4	36.1	41.5	50.7	69.0

Table 7. Instance semantic segmentation results (mIoU(%)) of different methods on the YCB-Video dataset. Jointly training semantic segmentation module with keypoint offset module ($\mathcal{M}_{S,K}$) obtains size information from the offset module and performs better, especially on large clamp and extra-large clamp. With the center voting module $\mathcal{M}_C$ and the Mean-Shift clustering algorithm, further improvement of performance is obtained.

space is bigger. Selecting 8 keypoints is a good trade-off.

Effect of Multi-task learning. In this part, we discuss how the joint learning of semantic segmentation and keypoint (center) translation offset boosts the performance. In Table 6, we explore how semantic segmentation enhances keypoint offset learning. We remove semantic segmentation and center voting modules $\mathcal{M}_S$, $\mathcal{M}_C$, and train our keypoint voting module $\mathcal{M}_K$ individually. During inference time, the instance semantic segmentation predicted by Mask R-CNN [15] ($\mathcal{M}_K$ +MRC) and the ground truth ($\mathcal{M}_K$ +GT) are applied. Experimental results show that jointly trained with semantic segmentation ($\mathcal{M}_{K,S}$ +GT) boosts the performance of keypoint offset voting and improves the accuracy

of 6D pose estimation by 0.7% on ADD(S) metric. We believe that the semantic module extracts global and local features to distinguish different objects. Such features also help the model to recognize which part of an object a point belongs to and improve offset prediction.

In Table 7, we explore how keypoint and center point offset learning improve the instance semantic segmentation result. Point mean intersection over union (mIoU) is used as evaluation metric. We report the results of the challenging large clamp and extra-large clamp in YCB-Video dataset. They look same in appearance but are different in size, as shown in Figure 5. We trained Mask R-CNN (ResNeXt-50-FPN) [15] with the recommended setting as a simple baseline and found it was completely confused by the two objects. With extra depth information, our semantic segmentation module (PVN3D($\mathcal{M}_S$)), trained individually, didn’t perform well either. However, jointly trained with our keypoint offset voting module (PVN3D($\mathcal{M}_{S,K}$)), the mIoU was improved by 9.2% on the extra-large clamp. With voted centers obtained from the center voting module $\mathcal{M}_C$, we can split up objects with the Mean-Shift clustering algorithm and assign points to its closest object cluster. The mIoU of the extra-large clamp is further improved by 18.3% in this way. Some qualitative results are shown in Figure 5.

5. Conclusion

We propose a novel deep 3D keypoints voting network with instance semantic segmentation for 6DoF pose estimation, which outperforms all previous approaches in several datasets by large margins. We also show that jointly training 3D keypoint with semantic segmentation can boost the performance of each other. We believe the 3D keypoint based approach is a promising direction to explore for the 6DoF pose estimation problem.

A. Appendix

A.1. Architecture Details.

The image embedding network in the Feature Extraction module consists of a standard ResNet-34 encoder pretrained with ImageNet and followed by 4 up-sampling layers as a decoder. The output RGB feature is of 128 channels. The point cloud embedding network is a PointNet++ [40] with Multi-scale Grouping (MSG), the output of which is also of 128 channels. In the DenseFuion block, for each sampled point, the 128 channels RGB feature is concatenated with the corresponding 128 channels point cloud feature and is fed into shared MLPs to raise the feature channels to 1024. The 128 channels RGB and point cloud feature are also fed into shared MLPs and raised to 256 channels respectively. They are then all concatenated together to form the 1792 ($128*2+256*2+1024$) channels RGBD features. The 3D keypoint offset voting module $\mathcal{M}_K$ consists of shared MLPs ($1792-1024-512-128-N_{kp}*3$) reducing the 1792 channels RGBD features to N_{kp} 3D keypoints offset. The semantic segmentation module $\mathcal{M}_S$ consists of shared MLPs ($1792-1024-512-128-N_{cls}$) with N_{cls} the number of object classes. The center voting module $\mathcal{M}_C$ also consists of shared MLPs ($1792-1024-512-128-3$). During training, Adam optimization algorithm is employed, with a mini-batch size of 24. We applied cyclical learning rates during training, the range of which is from $1e-5$ to $1e-3$.

A.2. Implementation: Models for LineMOD dataset.

In the LineMOD dataset, though there are multi objects in one scene, only the label of one target object in each scene is provided, making it hard to train a single model for all objects. Therefore, we follow PVNet[37] and trained models separately for each object. It means our semantic segmentation module $\mathcal{M}_S$ only predicts two semantic labels for each object, label of the target object and the background.

A.3. Implementation: PVN3D+ICP

We introduce our implementation of PVN3D+ICP as follows. During training and inference of PVN3D, we randomly sample 12288 points (pixels) from the whole scene as input. Only these points are labeled with the semantic label by our instance semantic segmentation module, which is not enough for a good performance of the ICP algorithm. Therefore, for each unlabeled point in the whole point cloud scene, we find its closest labeled point and assigned that label to it. Points with the same instance semantic labels are then selected. To eliminate the effect of noise points, a MeanShift [7] clustering algorithm is also applied. The biggest cluster of points is then selected as the input of the ICP algorithm.

A.4. More Results

A.4.1 Instance semantic segmentation

We provide more results of instance semantic segmentation in Table 8.

A.4.2 Visualization of detected keypoints

The visualization of some detected keypoints is shown in Figure 6.

A.4.3 Evaluation on the Occlusion LineMOD Dataset

The Occlusion LineMOD dataset [3] is additionally annotated from the LineMOD datasets, objects of which are heavily occluded, making it harder for pose estimation. Evaluation results in Table 9 show that our PVN3D outperforms previous methods by a large margin.

A.4.4 Inference time

It takes 0.17 seconds for network forward propagation and 0.02 seconds for pose estimation of each object during inference. The overall runtime is 5 FPS on the LineMOD dataset.

References

- [1] K. S. Arun, T. S. Huang, and S. D. Blostein. Least-squares fitting of two 3-d point sets. *IEEE Transactions on pattern analysis and machine intelligence*, (5):698–700, 1987. 4
- [2] H. Bay, T. Tuytelaars, and L. Van Gool. Surf: Speeded up robust features. In *European conference on computer vision*, pages 404–417. Springer, 2006. 2
- [3] E. Brachmann, A. Krull, F. Michel, S. Gumhold, J. Shotton, and C. Rother. Learning 6d object pose estimation using 3d object coordinates. In *European conference on computer vision*, pages 536–551. Springer, 2014. 2, 9
- [4] B. Calli, A. Singh, A. Walsman, S. Srinivasa, P. Abbeel, and A. M. Dollar. The ycb object and model set: Towards common benchmarks for manipulation research. In *2015 international conference on advanced robotics (ICAR)*, pages 510–517. IEEE, 2015. 4
- [5] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia. Multi-view 3d object detection network for autonomous driving. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1907–1915, 2017. 1
- [6] A. Collet, M. Martinez, and S. S. Srinivasa. The moped framework: Object recognition and pose estimation for manipulation. *The International Journal of Robotics Research*, 30(10):1284–1306, 2011. 1
- [7] D. Comaniciu and P. Meer. Mean shift: A robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, (5):603–619, 2002. 3, 9

	PoseCNN [52]	Mask R-CNN [15]	$\mathcal{M}_S$	$\mathcal{M}_{S,\mathcal{K}}$	$\mathcal{M}_{S,\mathcal{K},C}$
002_master_chef_can	86.9	87.7	88.0	88.1	87.7
003_cracker_box	87.5	85.9	88.0	88.2	88.3
004_sugar_box	92.0	91.1	92.3	91.4	91.5
005_tomato_soup_can	86.4	87.1	88.5	88.7	88.7
006_mustard_bottle	93.1	93.1	92.7	93.2	93.3
007_tuna_fish_can	89.2	88.8	89.2	89.5	89.6
008_pudding_box	73.9	82.5	86.7	90.7	90.8
009_gelatin_box	84.3	91.9	92.6	89.5	89.5
010_potted_meat_can	83.9	84.9	88.4	86.4	86.3
011_banana	90.0	90.2	89.0	89.0	89.1
019_pitcher_base	95.7	92.2	96.1	96.3	96.4
021_bleach_cleanser	88.2	90.2	87.1	91.8	91.7
024_bowl	90.3	90.8	92.1	79.0	78.8
025_mug	83.0	80.7	83.8	84.1	84.1
035_power_drill	86.4	87.8	87.2	88.8	88.8
036_wood_block	79.8	83.2	82.6	78.5	78.5
037_scissors	65.7	51.6	67.1	77.1	77.2
040_large_marker	69.7	73.4	76.9	74.5	74.6
051_large_clamp	43.1	48.4	58.5	62.5	70.2
052_extra_large_clamp	30.4	36.1	41.5	50.7	69.0
061_foam_brick	87.7	87.4	89.3	87.4	87.3
MEAN	80.3	81.2	83.7	84.1	85.3

Table 8. Instance semantic segmentation results (mIoU(%)) of different approaches.

	PoseCNN [52]	Oberweger [34]	Hu et al. [20]	Pix2Pose [35]	PVNet [37]	DPOD [54]	PVN3D
ape	9.6	12.1	17.6	22.0	15.8	-	33.9
can	45.2	39.9	53.9	44.7	63.3	-	88.6
cat	0.9	8.2	3.3	22.7	16.7	-	39.1
driller	41.4	45.2	62.4	44.7	65.7	-	78.4
duck	19.6	17.2	19.2	15.0	25.2	-	41.9
eggbox	22.0	22.1	25.9	25.2	50.2	-	80.9
glue	38.5	35.8	39.6	32.4	49.6	-	68.1
holepuncher	22.1	36.0	21.3	49.5	39.7	-	74.7
average	24.9	27.0	27.0	32.0	40.8	47.3	63.2

Table 9. Quantitative evaluation of 6D Pose on ADD(S) [19] metric on the Occlusion LineMOD dataset. Objects with bold name are symmetric.

- [8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 4
- [9] A. Doumanoglou, R. Kouskouridas, S. Malassiotis, and T.-K. Kim. Recovering 6d object pose and predicting next-best-view in the crowd. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3583–3592, 2016. 2
- [10] K. Duan, S. Bai, L. Xie, H. Qi, Q. Huang, and Q. Tian. Centernet: Keypoint triplets for object detection. *arXiv preprint arXiv:1904.08189*, 2019. 4
- [11] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3354–3361. IEEE, 2012. 1
- [12] D. Glasner, M. Galun, S. Alpert, R. Basri, and G. Shakhnarovich. aware object detection and pose estimation. In *2011 International Conference on Computer Vision*, pages 1275–1282. IEEE, 2011. 2
- [13] C. Gu and X. Ren. Discriminative mixture-of-templates for viewpoint classification. In *European Conference on Computer Vision*, pages 408–421. Springer, 2010. 2
- [14] K. Gupta, L. Petersson, and R. Hartley. Cullnet: Calibrated and pose aware confidence scores for object pose estimation. In *Proceedings of the IEEE International Conference on Computer Vision Workshops*, pages 0–0, 2019. 2
- [15] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 8, 10
- [16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE con-*

- [25] J. Ku, A. Harakeh, and S. L. Waslander. In defense of classical image processing: Fast depth completion on the cpu. In *2018 15th Conference on Computer and Robot Vision (CRV)*, pages 16–22. IEEE, 2018. 5
- [26] Y. Li, G. Wang, X. Ji, Y. Xiang, and D. Fox. Deepim: Deep iterative matching for 6d pose estimation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 683–698, 2018. 2, 6
- [27] Z. Li, G. Wang, and X. Ji. Cdpn: Coordinates-based disentangled pose network for real-time rgb-based 6-dof object pose estimation. In *The IEEE International Conference on Computer Vision (ICCV)*, October 2019. 2, 6
- [28] J. Liebelt, C. Schmid, and K. Schertler. Independent object class detection using 3d feature maps. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8. IEEE, 2008. 2
- [29] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017. 4
- [30] D. G. Lowe. Object recognition from local scale-invariant features. In *Proceedings of the seventh IEEE international conference on computer vision*, volume 2, pages 1150–1157. Ieee, 1999. 1, 2
- [31] E. Marchand, H. Uchiyama, and F. Spindler. Pose estimation for augmented reality: a hands-on survey. *IEEE transactions on visualization and computer graphics*, 22(12):2633–2651, 2015. 1
- [32] F. Michel, A. Kirillov, E. Brachmann, A. Krull, S. Gumhold, B. Savchynskyy, and C. Rother. Global hypothesis generation for 6d object pose estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 462–471, 2017. 2
- [33] A. Newell, K. Yang, and J. Deng. Stacked hourglass networks for human pose estimation. In *European conference on computer vision*, pages 483–499. Springer, 2016. 2, 7
- [34] M. Oberweger, M. Rad, and V. Lepetit. Making deep heatmaps robust to partial occlusions for 3d object pose estimation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 119–134, 2018. 2, 7, 10
- [35] K. Park, T. Patten, and M. Vincze. Pix2pose: Pixel-wise coordinate regression of objects for 6d pose estimation. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 7668–7677, 2019. 2, 10
- [36] G. Pavlakos, X. Zhou, A. Chan, K. G. Derpanis, and K. Daniilidis. 6-dof object pose from semantic keypoints. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2011–2018. IEEE, 2017. 1
- [37] S. Peng, Y. Liu, Q. Huang, X. Zhou, and H. Bao. Pvnnet: Pixel-wise voting network for 6dof pose estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4561–4570, 2019. 1, 2, 4, 5, 6, 7, 9, 10
- [38] C. R. Qi, O. Litany, K. He, and L. J. Guibas. Deep hough voting for 3d object detection in point clouds. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 9277–9286, 2019. 2, 4, 6
- [39] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas. Frustum pointnets for 3d object detection from rgb-d data. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 918–927, 2018. 2, 3, 4
- [40] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in neural information processing systems*, pages 5099–5108, 2017. 4, 9
- [41] M. Rad and V. Lepetit. Bb8: A scalable, accurate, robust to partial occlusion method for predicting the 3d poses of challenging objects without using depth. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 3828–3836, 2017. 1, 2
- [42] F. Rothganger, S. Lazebnik, C. Schmid, and J. Ponce. 3d object modeling and recognition using local affine-invariant image descriptors and multi-view spatial constraints. *International Journal of Computer Vision*, 66(3):231–259, 2006. 2
- [43] H. Su, C. R. Qi, Y. Li, and L. J. Guibas. Render for cnn: Viewpoint estimation in images using cnns trained with rendered 3d model views. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2686–2694, 2015. 2
- [44] M. Sun, G. Bradski, B.-X. Xu, and S. Savarese. Depth-encoded hough voting for joint object detection and shape recovery. In *European Conference on Computer Vision*, pages 658–671. Springer, 2010. 2
- [45] M. Sundermeyer, Z.-C. Marton, M. Durner, M. Brucker, and R. Triebel. Implicit 3d orientation learning for 6d object detection from rgb images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 699–715, 2018. 2, 6
- [46] S. Suwajanakorn, N. Snavely, J. J. Tompson, and M. Norouzi. Discovery of latent 3d keypoints via end-to-end geometric reasoning. In *Advances in Neural Information Processing Systems*, pages 2059–2070, 2018. 2
- [47] B. Tekin, S. N. Sinha, and P. Fua. Real-time seamless single shot 6d object pose prediction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 292–301, 2018. 1, 2
- [48] J. Tremblay, T. To, B. Sundaralingam, Y. Xiang, D. Fox, and S. Birchfield. Deep object pose estimation for semantic robotic grasping of household objects. *arXiv preprint arXiv:1809.10790*, 2018. 1
- [49] S. Tulsiani and J. Malik. Viewpoints and keypoints. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1510–1519, 2015. 2
- [50] C. Wang, D. Xu, Y. Zhu, R. Martín-Martín, C. Lu, L. Fei-Fei, and S. Savarese. Densefusion: 6d object pose estimation by iterative dense fusion. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3343–3352, 2019. 1, 2, 3, 4, 5, 6
- [51] H. Wang, S. Sridhar, J. Huang, J. Valentin, S. Song, and L. J. Guibas. Normalized object coordinate space for category-level 6d object pose and size estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2642–2651, 2019. 2

- [52] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. *arXiv preprint arXiv:1711.00199*, 2017. [1](#), [2](#), [5](#), [6](#), [8](#), [10](#)
- [53] D. Xu, D. Anguelov, and A. Jain. Pointfusion: Deep sensor fusion for 3d bounding box estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 244–253, 2018. [1](#), [2](#), [3](#), [4](#)
- [54] S. I. Zakharov, S. and S. Ilic. Dpod: 6d pose object detector and refiner. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1941–1950, 2019. [10](#)
- [55] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia. Pyramid scene parsing network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2881–2890, 2017. [4](#)
- [56] M. Zhu, K. G. Derpanis, Y. Yang, S. Brahmabhatt, M. Zhang, C. Phillips, M. Lecce, and K. Daniilidis. Single image 3d object detection and pose estimation for grasping. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3936–3943. IEEE, 2014. [1](#)